Automotive Safety Gateway

QNX RTOS 8.0 on Raspberry Pi 4

Complete Bring-Up Report: USB SSD Boot, UART Console, STM32 Edge Node Integration,

VFS Resource Manager, UDP Telemetry Bridge, and ASIL Watchdog

Document ID: ASG-QNX-RPI4-003 | Revision: C (Phase 4 Complete)

Pratik Suryawanshi
pratik.suryawanshi@uni-bayreuth.de

April 21, 2026

Abstract

This comprehensive engineering report documents the complete bring-up of an Automotive Safety Gateway (ASG) prototype. The system comprises a Raspberry Pi 4 Model B running QNX RTOS 8.0.0 (AArch64) as the Zonal Controller, booting from an external USB 3.0 SSD, and an STM32F411CEU6 microcontroller (bare-metal C) serving as a deterministic Edge Node. The document covers four critical phases: (0) hardware bootstrap and USB boot configuration, (1) QNX console bring-up via miniUART, (2) establishment of a robust, ASIL-aware inter-processor communication (IPC) link over UART3 using a custom binary protocol, and (4) integration of a POSIX VFS Resource Manager, UDP network telemetry bridge, and hardware watchdog for fault tolerance. Every register-level configuration, driver quirk, and architectural pivot is recorded with root-cause analysis. This document serves as the definitive reference for hardware-software interface (HSI) and future production hardening.

Revision	Date	Author	Description
A	2026-04-16	P. Suryawanshi	Initial bring-up: Phases 0–2 documented.
B	2026-04-20	P. Suryawanshi	Added Phase 4 (VFS, UDP bridge, IWDG).
C	2026-04-21	P. Suryawanshi	Incorporated hardware photo and final edits.

Contents

Hardware Setup **4**

1 System Architecture Overview **5**

2 Phase 0: Hardware Bootstrap & USB SSD Payload Staging **5**
2.1 Base OS Initialization . . . 5
2.2 Firmware UAS Quirk Application . . . 5
2.3 EEPROM Update (USB Boot Enablement) . . . 5
2.4 QNX Payload Staging . . . 6
2.5 Handoff Preparation . . . 6

3 Phase 1: QNX Console Bring-Up via miniUART **6**
3.1 FTDI Debug Wiring . . . 6
3.2 Bootloader Configuration . . . 6
3.3 QNX Kernel Startup Arguments . . . 7
3.4 Serial Driver Initialization in Buildfile . . . 7
3.5 USB Mass Storage & UAS Quirk Bypassing . . . 7

4 Phase 2: STM32 Edge Node to QNX Gateway IPC Link **7**
4.1 Edge Node Implementation (STM32F411CEU6 – Bare-Metal C) . . . 7
4.1.1 Clock Tree Configuration . . . 7
4.1.2 USART2 Pin Multiplexing (PA2 as TX) . . . 8
4.1.3 Baud Rate Calculation (115200 @ 16 MHz) . . . 8
4.1.4 Soft-Float ABI Linkage Fix . . . 8
4.2 QNX Zonal Controller Configuration (PL011 UART3) . . . 8
4.2.1 Device Tree Overlay for UART3 . . . 8
4.2.2 PL011 Driver Initialization and GIC Vector Mapping . . . 9
4.2.3 Hardware Wiring: STM32 to Raspberry Pi . . . 9
4.3 QNX User-Space Ingestion Daemon (`gateway_rm.c`) . . . 9

5 ASIL-Compliant Binary Protocol (Layer 2) **10**
5.1 Frame Structure (`#pragma pack(1)`) . . . 10
5.2 STM32 Frame Assembly (Bare-Metal) . . . 10
5.3 QNX Finite State Machine (FSM) Receiver . . . 10

6 Phase 4: Distributed Systems & VFS Integration **11**
6.1 QNX POSIX VFS Resource Manager . . . 11
6.1.1 Implementation Architecture . . . 11
6.1.2 Resource Manager Skeleton Code . . . 12
6.2 Network Telemetry Bridge (UDP Broadcasting) . . . 13
6.2.1 Implementation Architecture . . . 13
6.3 ASIL Fault-Tolerance: STM32 Independent Watchdog (IWDG) . . . 13
6.3.1 Timeout Calculation . . . 14
6.3.2 Register Initialization Sequence . . . 14
6.3.3 Kicking the Watchdog in Main Loop . . . 14
6.3.4 QNX Fault Handling . . . 14

7 Known Architectural Deviations from Production Intent **15**

8 Conclusion **15**

Hardware Setup

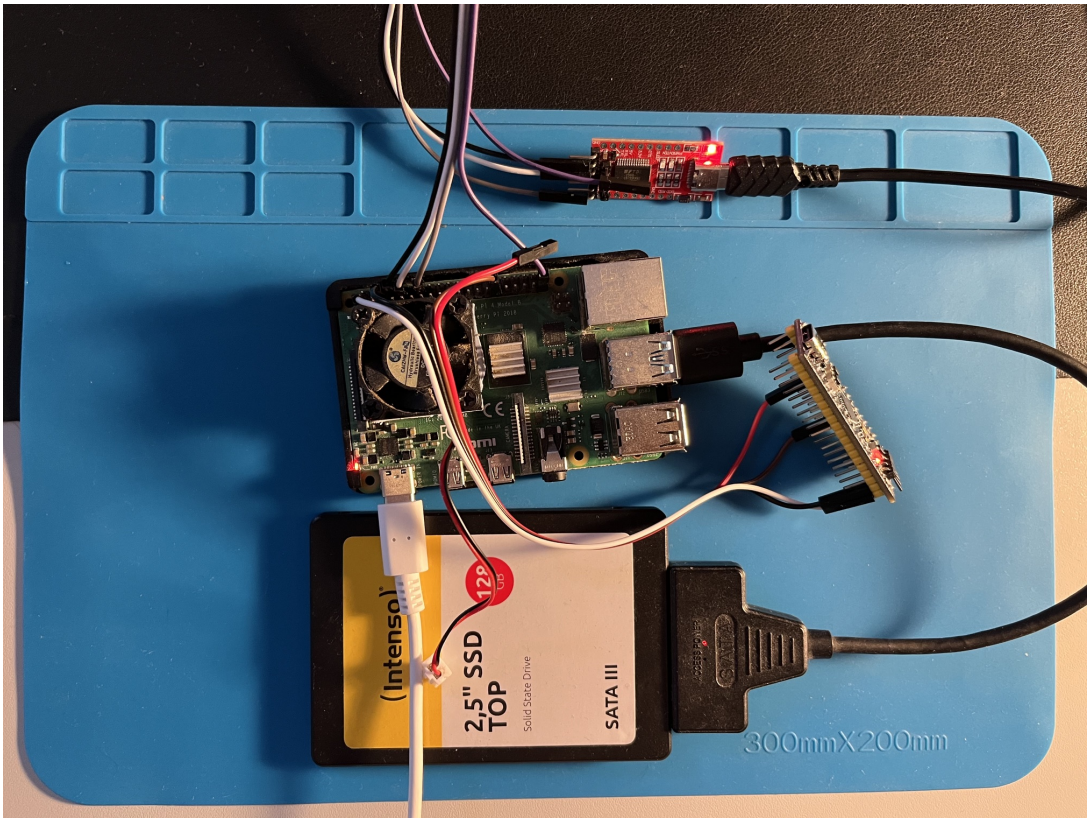

Figure 1: Physical hardware configuration: Raspberry Pi 4 (QNX Zonal Controller) with USB SSD attached, FTDI serial debugger connected to GPIO 14/15, and STM32F411 Edge Node wired to GPIO 4/5 for UART3 communication.

1 System Architecture Overview

Table 1: Platform Component Summary

Component	Specification
Zonal Controller (SoC)	Broadcom BCM2711 (Raspberry Pi 4 Model B)
Root Operating System	QNX RTOS 8.0.0 (AArch64)
Boot Media	External USB 3.0 SSD (JMicron JMS578 bridge)
Console Interface	miniUART on GPIO 14/15 (FTDI 3.3V)
Edge Node (MCU)	STM32F411CEU6 (ARM Cortex-M4)
Edge Node Runtime	Bare-metal C (no HAL, direct register access)
IPC Physical Layer	UART3 (PL011) on GPIO 4/5 @ 115200 8N1
VFS Device Node	`/dev/asg_sensor` (Resource Manager)
Network Telemetry	UDP broadcast over Ethernet
Safety Watchdog	STM32 Independent Watchdog (IWDG) @ 1.0 s timeout

The system implements an Asymmetric Multi-Processing (AMP) architecture. The QNX Zonal Controller provides a POSIX-compliant resource manager, while the STM32 Edge Node simulates a deterministic sensor array transmitting temperature telemetry over a robust, checksummed binary protocol. Phase 4 extends this foundation with VFS abstraction, network visibility, and ASIL-oriented fault recovery.

2 Phase 0: Hardware Bootstrap & USB SSD Payload Staging

Before the QNX microkernel could execute, the Raspberry Pi 4's onboard EEPROM and bootloader required explicit configuration to boot from an external USB 3.0 SSD, bypassing native JMicron UAS conflicts. This phase was executed from a temporary Raspberry Pi OS environment.

2.1 Base OS Initialization

1. Boot the Pi 4 using a standard microSD card flashed with Raspberry Pi OS.
2. Connect the target SSD (via Ankeje/JMicron adapter) to a Pi USB 3.0 port.

2.2 Firmware UAS Quirk Application

The JMicron bridge (VID:`0x152d`, PID:`0x0576`) fails to enumerate correctly under UAS. To force the Linux kernel (and subsequent bootloader stages) to fall back to Bulk-Only Transport (BOT), append the following quirk to the front of `/boot/firmware/cmdline.txt`:

```
usb-storage.quirks=152d:0576:u
```

Listing 1: USB Storage Quirk in cmdline.txt

2.3 EEPROM Update (USB Boot Enablement)

Update the onboard EEPROM to the latest stable bootloader, enabling USB mass storage boot priority:

```
sudo rpi-eeprom-update -d -a
sudo reboot
```

Listing 2: Raspberry Pi EEPROM Update

2.4 QNX Payload Staging

Transfer the compiled QNX Initial File System (IFS) binary to the SSD's FAT32 boot partition and sync buffers:

```
sudo cp ~/ifs-rpi4.bin /boot/firmware/ifs-rpi4.bin
sudo sync
```

Listing 3: Staging the QNX IFS Payload

2.5 Handoff Preparation

Shut down Raspberry Pi OS cleanly. Remove the microSD card. The system is now staged to boot directly from the SSD using the custom `config.txt` defined in Section 3.2.

3 Phase 1: QNX Console Bring-Up via miniUART

This phase establishes the primary development console on the Pi's secondary UART (miniUART), avoiding contention with the Bluetooth subsystem.

3.1 FTDI Debug Wiring

Table 2: FTDI to Raspberry Pi GPIO miniUART Connection

Raspberry Pi	Signal	FTDI USB-TTL
Pin 6 (GND)	GND	GND
Pin 8 (GPIO 14 / TXD)	TXD	RX
Pin 10 (GPIO 15 / RXD)	RXD	TX

- **Host PC Configuration:** `minicom` @ 115200 baud, 8N1, Hardware Flow Control **OFF**.

3.2 Bootloader Configuration

The file `/boot/firmware/config.txt` is modified to lock the VPU core clock and route the miniUART.

```
# ==========================================
# QNX 8.0 AMP Platform Configuration
# ==========================================
# 1. Force AArch64 execution
arm_64bit=1

# 2. Enable miniUART and route it to GPIO 14/15
enable_uart=1

# 3. Lock the VPU core clock to 500MHz to stabilize miniUART baud rate
core_freq=500
core_freq_min=500

# 4. Show bootloader logs before kernel handoff (Debug only)
uart_2ndstage=1

# 5. Boot payload handoff
kernel=ifs-rpi4.bin
```

Listing 4: /boot/firmware/config.txt

Note: The directive `dtoverlay=disable-bt` is *not* used, as it would forcibly route the PL011 UART to GPIO 14/15, causing a pinmux conflict with our miniUART configuration.

3.3 QNX Kernel Startup Arguments

The QNX startup arguments explicitly bind the miniUART base address, reference clock, and baud rate.

```
[virtual=aarch64le,raw -compress] boot = {
    startup-bcm2711-rpi4 -v -D miniuart.0xfe215000.500000000.115200 -u reg -r 0,4096,1
    PATH=/proc/boot:/sbin:/bin:/usr/bin:/usr/sbin:/usr/libexec LD_LIBRARY_PATH=/proc/boot:/lib:/
        usr/lib:/lib/dll:/lib/dll/pci procnto-smp-instr -v
}
```

Listing 5: Kernel Startup Argument (excerpt)

3.4 Serial Driver Initialization in Buildfile

```
display_msg "Starting serial driver (/dev/ser1)"
devc-serminiuart -b115200 -c500000000 -e -F -u1 0xfe215000,125
waitfor /dev/ser1
reopen /dev/ser1
```

Listing 6: Serial Driver Initialization (Buildfile Snippet)

3.5 USB Mass Storage & UAS Quirk Bypassing

Under QNX, the JMicron bridge also requires UAS bypass. The `devb-umass` driver is started with the `prot=umass` flag to force Bulk-Only Transport.

```
echo "Starting USB Storage (UAS Bypass)..."
devb-umass cam pnp noslow umass prot=umass wait=30
waitfor /dev/hd0 30
```

Listing 7: USB Mass Storage Startup Script (usb_start.sh)

Important: QNX's CAM subsystem exposes USB block devices as `/dev/hd0` (hard disk) rather than `/dev/sd0`.

4 Phase 2: STM32 Edge Node to QNX Gateway IPC Link

This phase establishes a deterministic, ASIL-aware communication channel between the bare-metal STM32F411 sensor node and the QNX Zonal Controller via UART3 (PL011).

4.1 Edge Node Implementation (STM32F411CEU6 – Bare-Metal C)

To guarantee absolute execution determinism, the STM32 firmware uses direct memory-mapped register manipulation, completely bypassing the ST HAL libraries.

4.1.1 Clock Tree Configuration

The MCU is forced to run exclusively on the High-Speed Internal (HSI) oscillator at 16 MHz.

```
// Enable HSI
RCC->CR |= RCC_CR_HSION;
while (!(RCC->CR & RCC_CR_HSIRDY));

// Set HSI as system clock
RCC->CFGR &= ~RCC_CFGR_SW;
```

```
RCC->CFGR |= RCC_CFGR_SW_HSI;
while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_HSI);
```

Listing 8: STM32 Clock Initialization (Register Level)

4.1.2 USART2 Pin Multiplexing (PA2 as TX)

PA2 is configured for Alternate Function 7 (USART2_TX).

```
// Enable GPIOA and USART2 clocks
RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
RCC->APB1ENR |= RCC_APB1ENR_USART2EN;

// Configure PA2: Alternate Function mode, AF7 (USART2)
GPIOA->MODER &= ~GPIO_MODER_MODER2;
GPIOA->MODER |= GPIO_MODER_MODER2_1;            // 10 = AF mode
GPIOA->AFR[0] |= (7 << (2 * 4));                // AF7 for pin 2

// USART2: 8N1, enable transmitter
USART2->CR1 = USART_CR1_TE | USART_CR1_UE;
```

Listing 9: STM32 GPIO and USART2 Initialization

4.1.3 Baud Rate Calculation (115200 @ 16 MHz)

The USARTDIV is calculated as:

$$\text{USARTDIV} = \frac{16 \times 10^6}{16 \times 115200} = 8.6805$$

Mantissa = 8, Fraction = $0.6805 \times 16 \approx 11$.

```
USART2->BRR = (8 << 4) | 11;
```

Listing 10: STM32 Baud Rate Register Configuration

4.1.4 Soft-Float ABI Linkage Fix

Because the project uses `-nostdlib` to exclude standard C libraries, floating-point operations require explicit linking of the GCC software floating-point library to avoid undefined references to `__aeabi_fadd` and similar symbols.

```
arm-none-eabi-gcc -mcpu=cortex-m4 -mthumb -nostdlib -lgcc ...
```

Listing 11: STM32 Compilation Flags

4.2 QNX Zonal Controller Configuration (PL011 UART3)

4.2.1 Device Tree Overlay for UART3

To expose the PL011 UART3 on GPIO pins 4 (TXD) and 5 (RXD), the Broadcom bootloader must apply a device tree overlay. Because the root filesystem is read-only RAM, the `config.txt` file residing on the FAT32 boot partition must be modified directly.

```
mount -t dos /dev/hd0t12 /tmp
echo "dtoverlay=uart3" >> /tmp/config.txt
sync
umount /tmp
```

Listing 12: Mounting FAT32 Partition and Editing config.txt

4.2.2 PL011 Driver Initialization and GIC Vector Mapping

The `devc-serpl011` driver is started with the correct base address and interrupt vector. **Critical:** The ARM Generic Interrupt Controller (GIC-400) offsets Shared Peripheral Interrupts (SPI) by 32. The BCM2711 PL011 block uses SPI 121; therefore, the absolute GIC vector is $121 + 32 = 153$.

```
devc-serpl011 -e -b115200 -c48000000 0xfe201600,153
waitfor /dev/ser2 10
```

Listing 13: Starting devc-serpl011 with Correct GIC Vector

Flags:

- `-e`: Enable the receiver.
- `-b115200`: Baud rate.
- `-c48000000`: UART reference clock (48 MHz).
- `0xfe201600`: Base address of PL011 UART3.
- `153`: GIC interrupt number.

Note: The `-F` (hardware flow control) flag *must not* be used, as the physical connection lacks RTS/CTS lines. Using `-F` would deadlock the transmission FIFO.

4.2.3 Hardware Wiring: STM32 to Raspberry Pi

Table 3: STM32 UART2 to Raspberry Pi UART3 (PL011) Wiring

STM32F411 (PA2)	Signal	Raspberry Pi GPIO
PA2 (USART2_TX)	TXD	GPIO 4 (Pin 7)
GND	GND	GND (Pin 6 or 9)

Important: The STM32 TX connects to the Raspberry Pi RX (GPIO 4). No CTS/RTS wiring is present.

4.3 QNX User-Space Ingestion Daemon (`gateway_rm.c`)

A custom POSIX C application reads raw binary frames from `/dev/ser2` using `termios` in raw mode.

```c
#include <fcntl.h>
#include <termios.h>
#include <unistd.h>
#include <stdio.h>

int main() {
    int fd = open("/dev/ser2", O_RDONLY);
    struct termios tio;
    tcgetattr(fd, &tio);

    // Raw mode: 8N1, no canonical processing
    cfmakeraw(&tio);
    tio.c_cflag |= (CLOCAL | CREAD | CS8);
    tio.c_cflag &= ~PARENB;
    tio.c_cflag &= ~CSTOPB;
    tio.c_lflag &= ~(ICANON | ECHO | ECHOE | ISIG);
    tio.c_iflag &= ~(IXON | IXOFF | IXANY);

    cfsetispeed(&tio, B115200);
    cfsetospeed(&tio, B115200);
    tcsetattr(fd, TCSANOW, &tio);

    // ... frame processing (see Section 5) ...
```

```
    return 0;
}
```

Listing 14: QNX UART Receiver (termios Configuration)

5 ASIL-Compliant Binary Protocol (Layer 2)

Raw ASCII transmission proved vulnerable to EMI-induced framing errors. A deterministic binary protocol with synchronization, sequencing, and CRC-16 integrity checking was implemented.

5.1 Frame Structure (`#pragma pack(1)`)

Table 4: Binary Frame Format (8 bytes total)

Offset	Field	Type	Description
0	Sync1	`uint8_t`	0xAA (Start of frame)
1	Sync2	`uint8_t`	0x55 (Confirmation)
2	Sequence	`uint8_t`	Rolling counter (0–255)
3–6	Temperature	`float`	IEEE 754 single precision
7–8	CRC-16	`uint16_t`	CCITT-False (Polynomial 0x1021) over bytes 2–6

5.2 STM32 Frame Assembly (Bare-Metal)

```
typedef struct __attribute__((packed)) {
    uint8_t  sync1;
    uint8_t  sync2;
    uint8_t  seq;
    float    temp;
    uint16_t crc;
} sensor_frame_t;

sensor_frame_t frame = {
    .sync1 = 0xAA,
    .sync2 = 0x55,
    .seq   = seq_counter++,
    .temp  = read_temperature()
};
frame.crc = crc16_ccitt((uint8_t*)&frame.seq, 5);

// Transmit via USART2 (blocking)
for (int i = 0; i < sizeof(frame); i++) {
    while (!(USART2->SR & USART_SR_TXE));
    USART2->DR = ((uint8_t*)&frame)[i];
}
```

Listing 15: STM32 Binary Frame Transmission (Snippet)

5.3 QNX Finite State Machine (FSM) Receiver

The QNX daemon processes incoming bytes through a strict FSM to reject malformed frames.

```
enum state { HUNT_SYNC1, HUNT_SYNC2, READ_DATA };
enum state st = HUNT_SYNC1;
uint8_t buffer[8];
```

```
int idx = 0;

while (read(fd, &byte, 1) == 1) {
    switch (st) {
        case HUNT_SYNC1:
            if (byte == 0xAA) st = HUNT_SYNC2;
            break;
        case HUNT_SYNC2:
            st = (byte == 0x55) ? READ_DATA : HUNT_SYNC1;
            idx = 0;
            break;
        case READ_DATA:
            buffer[idx++] = byte;
            if (idx == 6) { // remaining bytes (seq + temp + crc)
                st = HUNT_SYNC1;
                if (validate_crc(buffer))
                    process_frame(buffer);
            }
            break;
    }
}
```

Listing 16: QNX Receiver State Machine (Pseudocode)

A CRC mismatch or sequence gap is logged as a fault, providing diagnostic data for system integrity monitoring.

6 Phase 4: Distributed Systems & VFS Integration

Phase 4 elevates the Zonal Controller from a closed-loop IPC silo into a fully distributed, POSIX-compliant enterprise node. It accomplishes three critical objectives:

1. Upgrading the internal QNX Message Passing server into a formal QNX Resource Manager, mounting the STM32 directly into the OS Virtual File System (VFS) as `/dev/asg_sensor`.
2. Developing a UDP network bridge to route ASG telemetry across standard Ethernet.
3. Implementing an Independent Hardware Watchdog (IWDG) on the STM32 to satisfy ASIL-oriented fault-tolerance requirements.

6.1 QNX POSIX VFS Resource Manager

Relying solely on `MsgSend()`/`MsgReceive()` restricts hardware access to applications explicitly programmed with custom `struct` definitions. A true QNX Resource Manager abstracts the hardware entirely, allowing any standard UNIX utility (e.g., `cat`, `grep`, or standard Python/C scripts) to interact with the edge node using standard `open()`, `read()`, and `write()` calls.

6.1.1 Implementation Architecture

- **VFS Registration:** The daemon utilizes `resmgr_attach()` to mount `/dev/asg_sensor` into the QNX `procnto` namespace.
- **Thread Pool:** Instead of manually spawning `pthread_create`, the daemon uses QNX's native `thread_pool_create()`. This allows the microkernel to dynamically scale the number of threads servicing `/dev/asg_sensor` based on CPU load and concurrent client requests.
- **I/O Callbacks (`io_func_funcs_t`):** Override the default POSIX `_IO_READ` and `_IO_WRITE` handlers.

- When a user-space client calls `read()`, the OS sends an `_IO_READ` message to our daemon. The custom callback fetches the Mutex-protected telemetry (Temperature, Drops, Link State), serializes it into a string or raw buffer, and returns it via `MsgReply()`.
- When a client calls `write("1")`, the `_IO_WRITE` callback intercepts it, translates it into the binary command `0xBB 0x66`, and transmits it to the STM32.

6.1.2 Resource Manager Skeleton Code

```
#include <sys/iofunc.h>
#include <sys/dispatch.h>

static resmgr_connect_funcs_t connect_funcs;
static resmgr_io_funcs_t io_funcs;
static iofunc_attr_t attr;

int my_read(resmgr_context_t *ctp, io_read_t *msg, iofunc_ocb_t *ocb) {
    // Fetch latest telemetry (protected by mutex)
    char buffer[128];
    snprintf(buffer, sizeof(buffer), "TEMP=%.2f SEQ=%u DROPS=%u\n",
             latest_temp, latest_seq, dropped_frames);
    MsgReply(ctp->rcvid, strlen(buffer), buffer, strlen(buffer));
    return (_RESMGR_NOREPLY);
}

\subsubsection{Write Handler Implementation}
The \texttt{\_IO\_WRITE} callback intercepts user-space write requests and translates them into
    binary commands for the STM32.

\begin{lstlisting}[language=C, caption={QNX Resource Manager Write Callback Skeleton}]
int my_write(resmgr_context_t *ctp, io_write_t *msg, iofunc_ocb_t *ocb) {
    char buf[16];
    size_t nbytes = min(msg->i.nbytes, sizeof(buf) - 1);
    resmgr_msgread(ctp, buf, nbytes, sizeof(msg->i));
    buf[nbytes] = '\0';

    // Example: writing "1" sends the command sequence 0xBB 0x66
    if (strcmp(buf, "1") == 0) {
        uint8_t cmd[] = {0xBB, 0x66};
        uart_send(cmd, sizeof(cmd));  // User-provided UART transmit
    }

    MsgReply(ctp->rcvid, nbytes, NULL, 0);
    return (_RESMGR_NOREPLY);
}

int main() {
    thread_pool_attr_t pool_attr;
    io_funcs.write = my_write;
    dispatch_t *dpp = dispatch_create();
    iofunc_func_init(_RESMGR_CONNECT_NFUNCS, &connect_funcs,
                     _RESMGR_IO_NFUNCS, &io_funcs);
    io_funcs.read = my_read;
    iofunc_attr_init(&attr, S_IFCHR | 0666, 0, 0);

    resmgr_attach(dpp, NULL, "/dev/asg_sensor", _FTYPE_ANY, 0,
                  &connect_funcs, &io_funcs, &attr);

    thread_pool_create(thread_pool_handle, &pool_attr);
    thread_pool_start(thread_pool_handle);
    // ... main loop handling UART ingest ...
}
```

Listing 17: QNX Resource Manager Skeleton (excerpt)

6.2 Network Telemetry Bridge (UDP Broadcasting)

Automotive gateways must route zonal data to the central vehicle compute layer. A lightweight POSIX network daemon (`net_bridge.c`) accomplishes this.

6.2.1 Implementation Architecture

- **Decoupled Ingestion:** Because the Resource Manager now exposes `/dev/asg_sensor`, the network bridge does not need to know about UART, CRCs, or state machines. It simply calls `open("/dev/asg_sensor", O_RDONLY)`.
- **Socket Layer:** Initializes a standard BSD datagram socket (`AF_INET`, `SOCK_DGRAM`).
- **Execution Loop:** Operating at 10 Hz, the daemon `read()`s the local VFS node, encapsulates the payload into a UDP packet, and executes `sendto()` targeting a specific broadcast IP or diagnostic dashboard endpoint.

```c
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int sensor_fd = open("/dev/asg_sensor", O_RDONLY);
    int sock = socket(AF_INET, SOCK_DGRAM, 0);

    struct sockaddr_in dest = {
        .sin_family = AF_INET,
        .sin_port = htons(9999),
        .sin_addr.s_addr = inet_addr("192.168.1.255")
    };

    char buf[256];
    while (1) {
        ssize_t n = read(sensor_fd, buf, sizeof(buf)-1);
        if (n > 0) {
            sendto(sock, buf, n, 0, (struct sockaddr*)&dest, sizeof(dest));
        }
        usleep(100000); // 10 Hz
    }
}
```

Listing 18: UDP Network Bridge Skeleton

Production Note: The destination address should be read from a configuration file (e.g., `/etc/asg/bridge.conf`) rather than hard-coded. A robust implementation would also support multicast groups and VLAN tagging for automotive Ethernet integration.

6.3 ASIL Fault-Tolerance: STM32 Independent Watchdog (IWDG)

To simulate ASIL safety constraints, the system must survive a catastrophic software hang (e.g., a corrupted pointer trapping the STM32 in a `HardFault` loop). The STM32 features an Independent Watchdog driven by a physically isolated Low-Speed Internal (LSI) clock at 32 kHz. If the main CPU fails to "kick" the dog before the countdown expires, the silicon forces a hard hardware reset.

6.3.1 Timeout Calculation

The timeout period T is defined by:

$$T = \frac{4 \times 2^{\text{PR}} \times \text{RLR}}{32000}$$

To achieve a ~ 1.0 second timeout, set $\text{PR} = 3$ (divider of 32) and $\text{RLR} = 1000$:

$$T = \frac{4 \times 2^3 \times 1000}{32000} = \frac{4 \times 8 \times 1000}{32000} = 1.0 \text{ second}$$

Note: The LSI clock is factory-calibrated to 32 kHz but exhibits a tolerance of $\pm 40\%$ over temperature and voltage (actual range: 17–47 kHz per STM32F411 datasheet). The 1.0 s timeout is nominal; worst-case window is 0.68–1.88 s. For production, the IWDG must be characterized across the operating envelope or replaced with an external precision watchdog.

6.3.2 Register Initialization Sequence

```
// Unlock IWDG registers by writing 0x5555 to KR
IWDG->KR = 0x5555;

// Set prescaler to 32 (PR = 3)
IWDG->PR = 3;

// Set reload value to 1000
IWDG->RLR = 1000;

// Load the reload value and start the watchdog (write 0xCCCC to KR)
IWDG->KR = 0xCCCC;
```

Listing 19: STM32 IWDG Configuration (Bare-Metal)

6.3.3 Kicking the Watchdog in Main Loop

Inside the 500 ms telemetry loop, the watchdog must be refreshed by writing `0xAAAA` to the key register:

```
while (1) {
    IWDG->KR = 0xAAAA;  // Refresh watchdog

    // ... read sensor, assemble frame, transmit ...

    delay_ms(500);
}
```

Listing 20: STM32 Watchdog Refresh

6.3.4 QNX Fault Handling

If the STM32 reboots, the sequence counter in the binary packet will abruptly reset to 0. The QNX Resource Manager's state machine detects this negative delta, logs a `[CRITICAL_FAULT] Edge Node Reboot Detected`, and passes this flag up the VFS to safety applications. This mechanism provides end-to-end visibility of edge node health.

7 Known Architectural Deviations from Production Intent

1. **Non-Automotive Boot Timing:** The 30-second `wait` state for USB SuperSpeed enumeration and block device ready is incompatible with ISO 26262 requirements (< 2 seconds). For production (Rev B), boot media will be migrated to industrial eMMC or SPI NOR flash.

2. **Volatile `/var` Mounting:** Until a QNX6 Power-Safe file system partition is formatted on the SSD, a RAM disk hosts `/var`. This allows networking and SSH daemons to start but results in non-persistent logs across reboots.

3. **Soft-Float ABI Dependency:** The STM32 firmware currently relies on software floating-point emulation (linked via `-lgcc`). A production build should enable the hardware FPU (CP10/CP11) in the startup code to improve determinism and reduce CPU load.

4. **Watchdog Granularity:** The IWDG timeout of 1.0 second is appropriate for prototype validation but may require tighter bounds (e.g., 100 ms) in a safety-critical deployment. The LSI clock tolerance ($\pm 40\%$) must also be characterized over temperature.

8 Conclusion

The Automotive Safety Gateway prototype has successfully achieved:

- Stable boot of QNX RTOS 8.0 from a USB 3.0 SSD, with UAS quirks applied at both the Linux bootloader and QNX driver levels.
- Functional debug console via the miniUART, avoiding Bluetooth conflicts.
- Reliable inter-processor communication between a bare-metal STM32F411 Edge Node and the QNX Zonal Controller over UART3.
- Implementation of a robust, ASIL-aware binary protocol with CRC-16 integrity checking and frame synchronization.
- A POSIX-compliant VFS Resource Manager exposing `/dev/asg_sensor`, enabling standard UNIX tooling to access edge telemetry.
- A UDP network bridge that broadcasts sensor data to external consumers over Ethernet.
- An Independent Watchdog on the STM32 with automatic fault detection and reporting on the QNX side.

All configuration artifacts, register-level settings, driver parameters, and Phase 4 architectural extensions have been documented exhaustively. This report serves as the definitive HSI reference for future hardware revisions and production hardening.

Document generated: April 21, 2026